

Hard-Coding agent.py in Python

A tool-calling agent that routes between calculators, inventory, and document search.

Project: logistics-rag-agent

File: agent.py

Role: LLM decides which tool; Python executes it

Unlike rag.py (always retrieve docs), the agent chooses: math tool, SKU lookup, or search_logistics_docs -- then answers from tool results.

1. Big picture

```
user question
|
v
create_agent(llm, tools, system_prompt)
|
+-- maybe tool_calls (EOQ / safety / SKU / docs)
|   |
|   v
|   ToolMessage results
|
v
final AIMessage content (no tool_calls) -> return as answer
```

WHY

Agent = policy (system prompt) + tools + chat model loop. Your job is to constrain the policy and extract the final text cleanly.

2. Build step by step

Step 1: Imports

```
from langchain.agents import create_agent
from langchain_core.messages import AIMessage, BaseMessage
from langchain_ollama import ChatOllama

from rag import CHAT_MODEL
from tools import get_tools
```

- create_agent -- modern LangChain helper that builds a tool-calling graph
- CHAT_MODEL imported from rag -- one place to change the default model name

WHY

Sharing CHAT_MODEL avoids agent answering with llama-X while rag uses llama-Y by accident.

Step 2: Hard-code the agent system prompt

```
AGENT_SYSTEM_PROMPT = """You are a logistics operations agent.

Decide which tool to use:
- eoq_calculator -- ...
- safety_stock_estimator -- ...
- lookup_sample_inventory -- ...
- search_logistics_docs -- ...

Rules:
- Prefer tools for calculations and SKU lookups; do not invent numbers.
- For document/policy questions, call search_logistics_docs and answer ONLY from that context.
- If document context is insufficient, say exactly: I don't know
- ...
"""
```

- Lists tools by name -- reinforces routing beyond tool docstrings alone
- Mirrors grounded 'I don't know' policy for doc questions

WHY

System prompt is the agent's constitution. Duplicate key rules here even if tools already document themselves -- models need redundant reminders under tool pressure.

Step 3: build_agent factory

```
def build_agent(*, model: str = CHAT_MODEL, temperature: float = 0.0):  
    llm = ChatOllama(model=model, temperature=temperature)  
    tools = get_tools()  
    return create_agent(  
        llm,  
        tools=tools,  
        system_prompt=AGENT_SYSTEM_PROMPT,  
    )
```

- temperature=0 -- more reliable tool choice and less creative math
- get_tools() at build time -- fresh list of decorated tool objects

WHY

Factory pattern: tests can build_agent(model='tiny') without editing globals.

Step 4: Extract the final assistant text

```
def _final_text(messages: list[BaseMessage]) -> str:
    for msg in reversed(messages):
        if isinstance(msg, AIMessage) and msg.content and not msg.tool_calls:
            if isinstance(msg.content, str):
                return msg.content
            return str(msg.content)
    return ""
```

- Walk reversed -- the last complete assistant message is usually the answer
- Skip AIMessages that still have tool_calls -- those are mid-plan, not final
- Handle non-str content via str(...) -- some models return structured content blocks

WHY

agent.invoke returns a full message transcript, not a single string. Without _final_text, app.py would print an ugly dict.

Step 5: ask_agent: one turn

```
def ask_agent(question: str, *, model: str = CHAT_MODEL) -> str:
    agent = build_agent(model=model)
    logger.debug("agent question: %s", question)
    result = agent.invoke(
        {"messages": [{"role": "user", "content": question}]}
    )
    messages = result.get("messages", [])
    # debug-log tool calls and ToolMessage previews...
    return _final_text(messages)
```

- Input format -- LangChain agent graph expects a messages list
- Debug loop -- with -v you see which tool fired and a short result preview
- Rebuild agent each call -- simple CLI; production might cache the graph

WHY

Logging tool calls is how you prove the agent used EOQ instead of inventing 183.7.

3. Practice

- Ask: 'EOQ for D=12000, S=100, H=2' -- expect eoq_calculator
- Ask: 'Is SKU-1002 below reorder?' -- expect lookup_sample_inventory
- Ask a policy question from your PDFs -- expect search_logistics_docs
- Ask something absent from docs -- expect 'I don't know'

4. Common failures

- Empty answer -- _final_text found no final AIMessage; inspect result['messages']
- Wrong tool -- strengthen AGENT_SYSTEM_PROMPT and tool docstrings
- create_agent import errors -- LangChain version mismatch; check installed package docs